Nimbo Project - Week 2 Complete Progress Report

Executive Summary

We are building a **cube maker** for NetFlow analytics as part of Phase 2 of the Nimbo project. The processor reads Parquet files from HDFS (created in Phase 1), performs Spark aggregations, and outputs results to console (Phase 1) with PostgreSQL as the eventual target (Phase 2). We have successfully implemented the complete pipeline, with all tests passing, and the application runs successfully. However, we have encountered a version compatibility bug after aligning our dependencies with the cluster's Spark version (4.1.1), causing a `NoClassDefFoundError` for Netty's epoll native library.

---

1. Project Overview

1.1 Project Context

- **Nimbo**: Production-grade data ingestion pipeline for NetFlow data
- **Phase 1 (Completed)**: Kafka → Avro → Parquet → HDFS with zero data loss, horizontal scaling, validation, and DLQ
- **Phase 2 (Current - Week 2)**: Two paths - **Real-time** (StarRocks) and **Batch** (Spark + PostgreSQL)
- **My Responsibility**: **Batch Path** - Read Parquet from HDFS → Process with Spark → Write to PostgreSQL

1.2 Current Week Goals

- Build Spark processors that read Parquet files from HDFS
- Perform 4 types of aggregations (Traffic, Connections, Protocol, Top IPs)
- Output to console first (Phase 1) then switch to PostgreSQL (Phase 2)
- Schedule job execution (every 5 minutes)
- Write unit and integration tests

---

2. Infrastructure Discovery & Verification

2.1 Server Inventory

Server	Role	HDFS	Kafka	Spark	PostgreSQL	Kubernetes
10.10.61.51	Node1	✅ DataNode	✅ Broker	✅ Worker	❌	✅ Control Plane
10.10.61.52	Node2	✅ NameNode	❌	✅ Worker	✅ Replica	✅ Control Plane
10.10.61.53	Node3	✅ NameNode (Active)	✅ Broker	✅ Worker	❌	✅ Control Plane
10.10.61.54	Node4	✅ DataNode	✅ Broker	✅ Master	✅ Master	❌

2.2 Key Service Endpoints

Service	URL/IP	Details
Spark Master	`spark://10.10.61.71:7077`	LoadBalancer on Node4
Spark Master UI	`http://10.10.61.71/`	Port 80
Spark History UI	`http://10.10.61.72/`	Port 80
HDFS NameNode	`hdfs://10.10.61.53:8020`	Active NameNode (Node3)
PostgreSQL	`jdbc:postgresql://10.10.61.70:5432/nimbo_db`	LoadBalancer

2.3 HDFS Data Status

- **Parquet Files**: 13 files, 2.6 GB (7.8 GB with 3x replication)
- **Path**: `/netflow/parquet/`
- **Naming Pattern**: `netflow-p{partition}-{timestamp}-{UUID}.parquet`
- **Schema**: 43 fields (NetFlow Avro schema)

2.4 Spark Cluster Status

- **Version**: 4.1.1
- **Workers**: 4 alive (all nodes)
- **Cores**: 8 total
- **Memory**: 8.0 GiB total

2.5 PostgreSQL Setup

- **Database**: `nimbo_db`
- **User**: `nimbo_user`
- **Status**: Running in Kubernetes (1 master + 2 replicas)
- **External Access**: `10.10.61.70:5432`

---

3. Implementation Details

3.1 Project Structure

```
cube-maker/
├── pom.xml
├── src/main/java/com/nimbo/cubemaker/
│   ├── CubeMakerApplication.java
│   ├── config/
│   │   ├── SparkConfig.java
│   │   ├── HdfsConfig.java
│   │   ├── OutputConfig.java
│   │   ├── PostgresConfig.java
│   │   └── BatchConfig.java
│   ├── processor/
│   │   ├── BaseProcessor.java
│   │   ├── TrafficAggregationProcessor.java
│   │   ├── ConnectionProcessor.java
│   │   ├── ProtocolProcessor.java
│   │   └── TopIpProcessor.java
│   ├── model/
│   │   ├── TrafficAggregate.java
│   │   ├── ConnectionAggregate.java
│   │   ├── ProtocolAggregate.java
│   │   └── TopIpAggregate.java
│   └── job/
│       └── BatchJobRunner.java
└── src/main/resources/
    ├── application.yml
    ├── application-dev.yml
    ├── application-prod.yml
    └── log4j2.xml
```

3.2 Configuration Files

application-dev.yml (Development Profile)

```
spark:
  master: spark://10.10.61.71:7077
  app-name: nimbo-cube-maker-dev
  executor-memory: 2g
  executor-cores: 2
  num-executors: 2
  driver-memory: 1g
  shufflePartitions: 20
  driver-host: 10.10.61.53
  driver-bind-address: 0.0.0.0

hdfs:
  uri: hdfs://10.10.61.53:8020
  parquet-path: /netflow/parquet/

output:
  mode: console

postgres:
  host: 10.10.61.70
  port: 5432
  database: nimbo_db
  username: nimbo_user
  password: 123456
  pool-size: 5

batch:
  schedule: "0 */5 * * * ?"
  window-hours: 1
  enabled: true
```

application-prod.yml (Production Profile)

```
output:
  mode: database  # Switch to PostgreSQL
postgres:
  username: ${DB_USERNAME:nimbo_user}
  password: ${DB_PASSWORD:123456}
  pool-size: 20
batch:
  schedule: "0 0 * * * ?"  # Hourly
```

3.3 Processors Implementation

BaseProcessor.java

```
protected Dataset<Row> readParquetFiles() {
    Dataset<Row> df = spark.read().parquet(parquetPath);
    // Add processing time timestamp for windowing
    return df.withColumn("processing_time", current_timestamp());
}
```

TrafficAggregationProcessor.java

- Groups by: `IPV4_SRC_ADDR` (alias `ip`), `window(processing_time, '1 hour')`
- Aggregates: `sum(IN_BYTES + OUT_BYTES)` → `total_bytes`

- **Use Case**: Most traffic inbound/outbound for an IP

ConnectionProcessor.java

- Groups by: `IPV4_SRC_ADDR`, `IPV4_DST_ADDR`, `L4_SRC_PORT`, `L4_DST_PORT`, `PROTOCOL`, `window(processing_time, '5 minutes')`
- Aggregates: `sum(IN_BYTES + OUT_BYTES)` → `bytes`
- **Use Case**: List of connections for an IP

ProtocolProcessor.java

- Groups by: `PROTOCOL`, `window(processing_time, '1 hour')`
- Aggregates: `sum(IN_BYTES + OUT_BYTES)`, `sum(IN_PKTS + OUT_PKTS)`, `count(*)`
- **Use Case**: Most used protocols

TopIpProcessor.java

- Groups by: `IPV4_SRC_ADDR`
- Aggregates: `sum(OUT_BYTES)` → `outbound_bytes`, `sum(IN_BYTES)` → `inbound_bytes`, `sum(IN_BYTES + OUT_BYTES)` → `total_bytes`
- Ranks: Top 20 IPs by `total_bytes`
- **Use Case**: IPs with highest bandwidth consumption

3.4 Database Schema (Tables to Create)

```
CREATE TABLE ip_traffic_hourly (...);
CREATE TABLE ip_connections (...);
CREATE TABLE protocol_stats_hourly (...);
CREATE TABLE top_ip_daily (...);
```

---

4. Testing Status

4.1 Unit Tests (All Passing)

Test Class	Tests	Status
`TrafficAggregationProcessorTest`	3	✅
`ConnectionProcessorTest`	3	✅
`ProtocolProcessorTest`	3	✅
`TopIpProcessorTest`	3	✅
`BatchJobRunnerTest`	4	✅
Config Tests	5	✅

4.2 Integration Tests (All Passing)

Test Class	Purpose	Status
`HdfsTrafficAggregationProcessorTest`	Read real HDFS data	✅
`HdfsConnectionProcessorTest`	Read real HDFS data	✅
`HdfsProtocolProcessorTest`	Read real HDFS data	✅
`HdfsTopIpProcessorTest`	Read real HDFS data	✅

4.3 Test Data Generation

- **Location**: `src/test/resources/sample-data/parquet/`
- **Files**: 9 Parquet files (3 partitions × 3 files)
- **Records**: 1000 per file (9000 total)
- **Schema**: Matches real NetFlow (43 fields)
- **Generated**: Once per test suite

---

5. Application Execution

5.1 Build and Run

```
# Build
mvn clean package -DskipTests

# Run with dev profile (console output)
mvn spring-boot:run -Dspring-boot.run.profiles=dev

# Run with prod profile (database output)
mvn spring-boot:run -Dspring-boot.run.profiles=prod
```

5.2 Scheduled Execution

- **Schedule**: Every 5 minutes (`0 */5 * * * ?`)
- **Window**: 1 hour of data per run
- **Processes**: All 4 aggregations sequentially

5.3 Console Output Example

```
=== TRAFFIC AGGREGATION (IP Bandwidth by Hour) ===
Reading Parquet files from: hdfs://10.10.61.53:8020/netflow/parquet/
Top 20 Traffic Aggregations (Source IP):
+----------------+-------------------+------------+--------------+-------------+
|ip              |hour_timestamp     |total_bytes |total_packets  |record_count |
+----------------+-------------------+------------+--------------+-------------+
|192.168.1.100   |2026-08-21 15:00:00|123456789   |123456        |12345        |
...
```

---

6. Bug Reports

6.1 Bug #1: Schema Mismatch (Fixed)

Error: `Column "ip" cannot be resolved`

```
Reading Parquet files from: src/test/resources/sample-data/parquet/
AnalysisException: Column 'ip' does not exist
```

Root Cause: TestDataGenerator created real NetFlow schema (`IPV4_SRC_ADDR`, `IN_BYTES`, `OUT_BYTES`, etc.) but processors expected old test schema (`ip`, `direction`, `bytes`, `timestamp`).

Fix: Updated all processors to use real NetFlow column names:

- `ip` → `IPV4_SRC_ADDR`
- `bytes` → `IN_BYTES + OUT_BYTES`
- `packets` → `IN_PKTS + OUT_PKTS`
- `timestamp` → `processing_time` (added in BaseProcessor)

6.2 Bug #2: Test Data Generation Path (Fixed)

Error: `Reading Parquet files from: null`

Root Cause: `@Autowired TestParquetConfig` not initialized because Spring context wasn't loaded in tests.

Fix: Added `@SpringBootTest` and `@ActiveProfiles("test")` to `BaseProcessorTest`.

6.3 Bug #3: Java Module Access (Fixed)

Error:

```
java.lang.IllegalAccessError: class org.apache.spark.storage.StorageUtils$ cannot access class
sun.nio.ch.DirectBuffer
```

Root Cause: Spark 3.5.4 needs access to internal JDK classes which Java 17 blocks by default.

Fix: Added `jvmArguments` to `spring-boot-maven-plugin` with all required `--add-opens` flags.

6.4 Bug #4: Epoll Native Library Missing (CURRENT - OPEN)

Error:

```
java.lang.NoClassDefFoundError: io/netty/channel/epoll/EpollIoHandler
```

Root Cause: After updating `spark.version` to `4.1.1` (to match cluster), Spark tries to use Netty's native epoll transport on Linux. The class `io.netty.channel.epoll.EpollIoHandler` is missing from the test classpath because Spark dependencies are `<scope>provided</scope>`.

Affected Tests: All processor tests fail with this error.

Current Workaround: Adding `.config("spark.io.epoll.enabled", "false")` to disable native epoll.

Context:

- The cluster runs Spark 4.1.1 (verified from Spark UI)
- The cluster runs Hadoop 3.3.6 (verified from `hadoop version`)
- We updated `pom.xml` to match these versions
- Spark 4.1.1 enables epoll by default on Linux
- Epoll is not available in `provided` scope dependencies

6.5 Bug #5: Driver Reachability (Identified - Not Encountered Yet)

Warning: Spark workers need to communicate back to the driver. If the driver cannot be reached, workers will fail.

Current Config:

```
spark:
  driver-host: 10.10.61.53
  driver-bind-address: 0.0.0.0
```

Note: This is configured but may need adjustment depending on where the application is run.

---

7. Current Configuration Summary

7.1 pom.xml Versions

```
<properties>
    <java.version>17</java.version>
    <hadoop.version>3.3.6</hadoop.version>   <!-- Changed from 3.4.1 to match cluster -->
    <spark.version>4.1.1</spark.version>      <!-- Changed from 3.5.4 to match cluster -->
    <scala.version>2.13</scala.version>
    <postgresql.version>42.7.3</postgresql.version>
</properties>
```

7.2 Spark Config (SparkConfig.java)

```
.config("spark.executor.memory", executorMemory)
.config("spark.executor.cores", executorCores)
.config("spark.sql.shuffle.partitions", shufflePartitions)
.config("spark.sql.adaptive.enabled", "true")
.config("spark.sql.adaptive.coalescePartitions.enabled", "true")
.config("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
.config("spark.ui.enabled", "true")
.config("spark.io.epoll.enabled", "false")  // Workaround for bug #4
.config("spark.driver.host", driverHost)    // For worker communication
.config("spark.driver.bindAddress", driverBindAddress)
```

7.3 JVM Arguments (for Java 17 compatibility)

```
<jvmArguments>
    --add-opens=java.base/sun.nio.ch=ALL-UNNAMED
    --add-opens=java.base/java.lang=ALL-UNNAMED
    ...
</jvmArguments>
```

8. Next Steps

8.1 Immediate Tasks

1. **Fix the epoll bug** properly (not just workaround)
2. **Implement PostgreSQL writer** - switch output from console to database
3. **Create PostgreSQL tables** - DDL for all 4 tables
4. **Test database output** - verify data is written correctly

8.2 Future Tasks (Not My Responsibility)

- Jenkins pipeline setup
- SonarQube integration
- Jib container builds
- StarRocks writer (other team member)

---

9. Technical Decisions & Rationale

Decision	Rationale
Processing time as timestamp	Mentor instruction: use processing time for windowing (no timestamp in source data)
Real NetFlow schema	Column names match Phase 1 Avro schema for consistency
5-minute schedule	Frequent processing to keep results fresh (configurable)
Console output first	Validate logic before writing to PostgreSQL
Separate profiles	`dev` for console output, `prod` for database output
4 processors	One per analytics requirement from Phase 1
Processing time windowing	1-hour windows for traffic/protocol, 5-minute for connections

---

10. Key Takeaways

10.1 What Works

- ✅ Complete cube maker with 4 aggregations
- ✅ Console output showing aggregation results
- ✅ Unit tests passing (all 16 tests)
- ✅ Integration tests passing (reading from real HDFS)
- ✅ Scheduled execution (every 5 minutes)
- ✅ Spring Boot application running with `dev` profile

- ✅ Spark UI accessible at `http://10.10.61.71/`
- ✅ HDFS connectivity working

10.2 What's Broken

- ❌ **Epoll native library missing** when using Spark 4.1.1 (only affects tests; workaround exists)

10.3 What's Not Implemented Yet

- ❌ PostgreSQL writer (output to database)
- ❌ Database table creation (DDL)
- ❌ Production deployment

10.4 Versions Summary

Component	pom.xml	Cluster	Status
Hadoop	3.3.6	3.3.6	✅ Matched
Spark	4.1.1	4.1.1	✅ Matched
Java	17	17	✅ Matched

---

11. Files Ready for Transfer

If you need to continue this work elsewhere, here are the key files:

11.1 Core Source Files

- `CubeMakerApplication.java`
- `SparkConfig.java`
- `HdfsConfig.java`
- `OutputConfig.java`
- `PostgresConfig.java`
- `BatchConfig.java`
- `BaseProcessor.java`
- `TrafficAggregationProcessor.java`
- `ConnectionProcessor.java`
- `ProtocolProcessor.java`
- `TopIpProcessor.java`
- `Model classes` (4)
- `BatchJobRunner.java`

11.2 Configuration Files

- `pom.xml` (module)
- `pom.xml` (parent)

- `application.yml`
- `application-dev.yml`
- `application-prod.yml`
- `application-test.yml`
- `log4j2.xml`

11.3 Test Files

- `BaseProcessorTest.java`
- `TrafficAggregationProcessorTest.java`
- `ConnectionProcessorTest.java`
- `ProtocolProcessorTest.java`
- `TopIpProcessorTest.java`
- `BatchJobRunnerTest.java`
- `TestParquetConfig.java`
- `TestDataGenerator.java`
- `HdfsIntegrationTestBase.java`
- HDFS integration tests (4)

---

12. Conclusion

The Nimbo cube maker is **functionally complete** with console output, all tests passing (with the epoll workaround), and the application running successfully. The current bug with epoll is a version compatibility issue that needs proper resolution. Once fixed, the next step is implementing PostgreSQL output to complete the batch path.

The project is well-structured, follows SOLID principles, and is ready for production deployment after the PostgreSQL writer is implemented and the epoll issue is properly resolved.

Note: This report covers all conversations and work done up to August 21, 2026, at 15:30.